Weather App Code Guide

A plain-language tour of the Internet Applications assignment

What this explains
How the browser, Vue, Express and OpenWeather work together, followed by a guided tour of the project files and the ideas used in them.

The app: search for a city, see today's weather and the next three forecast days, get packing advice and current air-quality information, and optionally compare two destinations.

Files covered

- Main app: server.js, public/index.html, public/app.js, public/style.css, public/weather-icons.svg
- Learning example: server-first-iteration.js
- Setup: package.json, .env.example, .gitignore and README.md

Contents

- 1. The request flow
- 2. Express setup and routes
- 3. Reusable city lookup
- 4. Daily summaries and packing
- 5. Air quality, errors and comparison
- 6–7. Vue state, requests and formatting
- 8–10. HTML page structure and comparison
- 11. CSS layout and responsive rules
- 12–13. SVG icons, learning server and setup
- 14. Key terms and troubleshooting

1. The whole app in one request

When a user searches for a city, this is the path the information takes:

```
City input in the browser
  → Vue stores the input in `city`
  → `searchWeather()` sends GET /api/weather?city=...
  → Express reads the query and checks it
  → OpenWeather Geocoding finds latitude and longitude
  → OpenWeather Forecast returns readings every three hours
  → the server groups readings into local calendar days
  → the server also requests current air quality
  → Express returns a smaller JSON object
  → Vue stores it in `result`
  → the HTML template updates to show the forecast
```

The browser does not call OpenWeather directly. It calls the app's own Express server using a relative URL such as `/api/weather`. The server keeps the OpenWeather key in its environment and makes the outside requests. Since the page and route are served by the same server, the browser sees them as the same origin.

The comparison follows the same flow, but starts two city lookups from the server at once. It uses `GET /api/compare` and returns both city objects plus comparison figures.

There is no database, login or saved history. Each request contains the city name it needs, and the server creates a response from the latest API data.

2. `server.js`: starting Express and defining routes

The first lines load two Node modules:

```
const express = require("express");
const path = require("node:path");
```

`require` is CommonJS syntax for loading a module. Express supplies the web-server functions. Node's `path` module safely constructs a filesystem path to the `public` folder.

```
const app = express();
const port = process.env.PORT || 3000;
const host = process.env.HOST || "127.0.0.1";
```

`app` is the Express application. It is where routes and middleware are registered. `process.env` contains environment variables. The server uses `PORT` or `HOST` if supplied, otherwise it listens locally on port 3000.

`GET /api/weather`

An Express route has a method, a path and a function. This route handles a browser request such as:

```
GET /api/weather?city=Dublin
```

`req` is the request coming in; `res` is the response the server sends back. Express puts query-string values in `req.query`, so `req.query.city` is the text after `city=`. The handler trims whitespace, rejects an empty or very long name with status 400, and checks whether `OPENWEATHER_API_KEY` exists. A missing key produces status 500.

The `try` block calls `getWeatherForCity(city, apiKey)`. It is asynchronous, so the route uses `await`. On success, `res.json(...)` serializes the returned JavaScript object as JSON. On failure, `sendApiError` chooses an HTTP status and a readable message.

`GET /api/compare`

This route accepts two city values. The browser sends repeated query parameters, for example:

```
GET /api/compare?cities=Dublin%2C%20IE&cities=Lisbon%2C%20PT
```

Repeated values are useful because a city plus country code contains a comma. `getRequestedCities` turns those values into two trimmed strings. The route checks that it received exactly two names, then starts both city lookups:

```
const destinations = await Promise.all(
  cities.map((city) => getWeatherForCity(city, apiKey))
);
```

`map` calls `getWeatherForCity` once for each city. `Promise.all` waits for both promises and returns their results in the same order. The two lookups can make network requests at the same time because neither destination depends on the other. This is concurrent I/O; it does not mean JavaScript calculations are running on two CPU cores.

If either lookup fails, `Promise.all` rejects and the route sends an error response. If both succeed, the response contains `destinations` and `comparison`.

Static files and starting the server

`app.use(express.static(path.join(__dirname, "public")))` tells Express to serve the page, JavaScript, CSS and SVG files from `public`. `__dirname` means the folder that contains `server.js`.

`app.listen(port, host, callback)` starts accepting browser requests. The callback prints the local address. `npm start` runs this file and asks Node to load `.env` first.

3. `getWeatherForCity`: one reusable lookup

Both routes use `getWeatherForCity`. The function returns the same weather object for a single city, so the compare route does not need a second copy of the lookup code.

Step 1: turn a city name into coordinates

OpenWeather's forecast endpoint needs latitude and longitude. The Geocoding endpoint accepts a place name instead. The server constructs a URL and adds its query parameters with `URLSearchParams`:

```
const locationUrl = new URL("https://api.openweathermap.org/geo/1.0/direct");
locationUrl.search = new URLSearchParams({
  q: city,
  limit: "1",
  appid: apiKey
});
```

`q` is the requested place, `limit` asks for one match, and `appid` is the private API key. `getOpenWeatherJson` fetches this URL. The first returned place contains its coordinates and location details. If there are no matches, `createRequestError` makes a 404 error with a message the user can understand.

Step 2: request the forecast

The server creates a Forecast URL with the returned coordinates and `units: "metric"`. This makes temperatures Celsius and wind speed metres per second. The API response contains a `list` of readings, usually one every three hours, plus a timezone offset for the city.

The raw readings are too detailed for the page. The server passes them to `makeDailySummaries`, then uses the first summary as `today` and the next three as `days`:

```
const today = dailySummaries[0];
const days = dailySummaries.slice(1, 4);
```

The table therefore shows the three forecast dates after today. If the API returns no usable forecast, the function raises a 502 error.

Step 3: request current air quality

Air pollution is requested separately using the same coordinates. This call is optional: it has its own `try/catch`. If it fails, the server still returns the weather forecast and records an `available: false` air-quality object with a message.

The object returned to Vue

The function returns a plain JavaScript object with these fields:

- `city`: the resolved name, state and country.
- `today`: today's local-date summary.
- `days`: up to three following daily summaries.
- `packing`: umbrella and temperature advice based on those days.
- `airQuality`: the current AQI and pollutant warnings, or an unavailable message.
- `hasThreeDays`: whether all three future days were returned.

The route sends this object as JSON. Vue does not need to understand OpenWeather's original response structure.

4. Daily summaries and packing rules

Grouping three-hour readings by local date

`makeDailySummaries(readings, timezoneOffset)` uses a `Map` called `daysByDate`. For each reading it adds the city's timezone offset to the Unix timestamp, converts that time to an ISO date, and puts the reading into the matching date group. This avoids grouping every place by UTC midnight when the app should show local days.

For each date group, the function calculates:

- minimum and maximum temperature across readings;
- average temperature and average wind speed;
- total rain from each reading's `rain.3h` value;
- whether rain is expected, based on rain volume or a Rain, Drizzle or Thunderstorm condition;
- an icon and short description from a representative reading near the middle of the day.

It returns one summary object per date. That summary is the small, predictable shape used by both the normal forecast and city comparison.

Temperature categories

`getTemperatureCategory` applies three simple rules to the daily average:

- Below 8°C is `Cold`.
- From 8°C through 24°C is `Mild`.
- Above 24°C is `Hot`.

The category is included in each daily summary so the template can display it and the packing function can use it.

Packing advice

`makePackingAdvice(days)` checks the categories for the next three days. If they vary, it says to pack layers. If they are all Cold or all Hot, it gives more specific clothing advice. Otherwise it recommends a light layer for Mild temperatures.

It also checks `rainExpected` for every day. If any day has rain, `umbrellaNeeded` is true and the UI says to bring an umbrella. Otherwise, it says that an umbrella is not needed. The function returns the advice and the category list; it does not create HTML.

5. Air quality, errors and the comparison rule

makeAirQualitySummary

The air-pollution response includes an overall AQI from 1 to 5 and pollutant concentrations in `components`. The function maps the AQI number to a label such as Good, Fair or Moderate.

The `pollutants` array is a small data table in JavaScript. Each entry gives a pollutant key (such as `o3`), a display name, concentration limits, and a short health explanation. The function filters out missing or non-numeric concentrations and reports only values at or above the first, Good-band limit. It then assigns a category by checking which limit the value is below. The pollutant warning level and the overall AQI are related readings, but they are calculated and displayed separately.

Error helpers

`getOpenWeatherJson(url)` uses Node's built-in `fetch` to contact OpenWeather. A network failure is marked as a network error. It tries to parse the response body as JSON. If OpenWeather returns a non-success status, the helper throws an error carrying that HTTP status.

`messageForApiError(error, dataName)` converts technical errors to short text: it recognises network failure, an invalid API key (401), rate limiting (429), and server failure (5xx). `createRequestError` attaches a status and a safe message for errors generated by this app, such as a city not found. `sendApiError` chooses the status and sends `{ error: message }` as JSON.

The client also checks the response's `content-type` in `readJsonResponse`. If it receives an HTML page instead of JSON, it explains that the Express server should be started or restarted. This helps identify cases where a static preview server is serving the page but not the API.

makeCityComparison

The comparison function receives the two completed city objects. For each city it calculates average temperature across the future days, counts rainy days, counts Mild days and reads the AQI if available.

The app uses this clear rule for a better-weather label: fewer rainy days wins; if rainy-day counts tie, more Mild days wins. If both counts tie, neither city is marked as a clear winner. The comparison also records the absolute difference between average temperatures.

Umbrella advice says neither, one or both cities may need an umbrella, based on rainy-day counts. For air quality, a higher AQI is worse. The app names a worse-air city only when both readings are available and their AQIs differ; otherwise it says the AQI is tied or unavailable. The function returns the metrics, winner indices and one-line verdict for Vue.

6. `public/app.js`: Vue state and the first search

This file creates one Vue app using the global `Vue` object loaded in `index.html`.

Reactive data

`data()` returns the values Vue should track:

- `city`, `loading`, `result` and `error` are for the first search.
- `comparisonCity`, `comparisonResult`, `comparisonError` and `comparing` are for the second search.

When one of these values changes, Vue updates the parts of the page that use it. The code does not manually find elements and replace their HTML.

Computed city label

`cityLabel` is computed from the resolved city name, state and country in `result`. A computed value is recalculated when the data it uses changes. This gives the interface a consistent label and lets the comparison request use the resolved first city.

`searchWeather()`

This method trims the input and stops if it is empty or another request is running. It sets `loading`, clears old errors and results, then calls:

```
fetch("/api/weather?city=" + encodeURIComponent(searchCity))
```

`encodeURIComponent` makes spaces, commas and other special characters safe inside one query value. `fetch` returns a Promise, so `await` waits for the server response without freezing the page. The method passes the response to `readJsonResponse`, checks `response.ok`, and stores the returned object in `result`.

If an error happens, the message goes into `error`. The `finally` block always sets `loading` back to false, even when the request failed.

`readJsonResponse(response)`

This helper reads the `content-type` response header. The server's API replies should be JSON. If the browser receives HTML instead, the helper throws a clearer error than the raw JSON parser message. Otherwise, it returns `response.json()`, which parses the JSON body into a JavaScript object.

7. `app.js`: comparison and display helpers

`compareCities()`

The comparison method checks that the first city has loaded and that the second input is not empty. It marks the comparison as loading, clears the previous result, and creates a `URLSearchParams` object.

It calls `params.append("cities", ...)` twice, once for each city. This creates two query values while still supporting city names such as `Dublin, IE`. The method fetches `/api/compare`, reads JSON, checks for an HTTP error, and stores the response in `comparisonResult`. `catch` sets `comparisonError`; `finally` turns off `comparing`.

Formatting methods

- `formatDate` displays a full weekday and date for the main forecast.
- `formatShortDate` uses a shorter weekday and date inside comparison cards.
- `weatherIcon` looks at OpenWeather's icon code. The first two characters identify the condition group; the last character indicates day or night for sun and partly cloudy icons. It returns the matching symbol name from `weather-icons.svg`.
- `formatNumber` uses `Intl.NumberFormat` for one decimal place at most. It cleans tiny values near zero so they display as zero.
- `formatTemperature` adds `°C` after formatting.
- `formatRainfall` displays zero as `0`, very small non-zero rain as `<0.1`, and otherwise uses `formatNumber`.
- `formatCityName` joins the city, state and country while skipping empty parts.

At the end, `app.mount("#app")` connects Vue to the element with `id="app"` in the HTML. Before this line, Vue has the state and methods; after it, Vue controls that part of the page.

8. `public/index.html`: the page structure

The document head

The top of the file declares an HTML5 document, English language, character encoding, and responsive viewport. The description and theme color are browser metadata. A Google Fonts stylesheet loads Karla. The local `style.css` controls the design. Vue and `app.js` are loaded with `defer`, so the browser parses the page before running them.

The app shell and first city

The `<div id="app">` is Vue's mount point. Inside it, `.weather-shell` has a sidebar and a main panel. The sidebar begins with a form:

- `<label for="city">` names the input for assistive technology.
- `v-model.trim="city"` keeps the input and Vue's `city` value in sync, trimming extra spaces.
- `@submit.prevent="searchWeather"` calls the method and prevents a normal form reload.
- `:disabled` disables Search while a request is active or the input is empty.
- `required`, `maxlength` and `autocomplete` are standard browser input settings.

When `result` exists, `v-if` displays today's resolved city, date, icon, average temperature, category and high/low values. If there is no result, `v-else` shows a short prompt.

Loading, error and empty states

The main panel has separate sections for loading, error, no search yet and successful results. `v-if` decides which section should exist. `role="alert"` marks an error for assistive technology, and `aria-live="polite"` announces the loading message without interrupting the user.

The error panel includes a Retry button. It calls the same `searchWeather` method, so there is only one path for making a weather request.

9. `index.html`: forecast and information cards

The three-day table

Inside the result state, the forecast section contains a table with columns for day, conditions, temperature, wind and rainfall. `v-for="day in result.days"` repeats one table row per summary. `:key="day.date"` gives Vue a stable identity for each row.

The cells use Vue interpolation such as `{{ formatTemperature(day.minTemperature) }}`. This displays a value without constructing HTML strings in JavaScript. The weather icon uses an SVG `<use>` element that refers to a symbol by name. A notice appears if the server could not provide all three days.

The table is inside `.table-scroll`; on a narrow screen it can be scrolled horizontally instead of forcing every column into a tiny width.

Packing and air-quality cards

The packing section displays umbrella advice and temperature advice from `result.packing`. A ternary expression chooses the umbrella text, and `:class` adds either `needs-umbrella` or `no-umbrella` to style it.

The air section has three states: air data unavailable, no pollutants above the Good limit, or a list of warnings. `v-else-if` and `v-else` ensure the matching state is shown. `v-for` repeats warning rows. Each warning shows the pollutant name, concentration, category and health-risk text supplied by the server.

10. `index.html`: the two-city comparison

The comparison section appears after packing and air quality. Its form binds the second input to `comparisonCity` and submits through `compareCities`. While the server is working, a loading message appears; if it fails, the error appears with `role="alert"`.

When `comparisonResult` exists, the page shows:

- the server's one-line weather, umbrella and air-quality verdict;
- the difference between average temperatures;
- two destination cards, repeated with `v-for`;
- each city's average temperature, rainy-day count and AQI;
- each available forecast day with an icon, condition, temperature range, wind and rainfall.

The `index` value in the loop identifies whether a card is the first or second destination. `:class` adds `comparison-card-winner` when that city's index matches `betterWeatherIndex`. A separate `v-if` displays the “Better forecast” label. Another conditional marks the city with worse air when the AQI comparison is possible.

If either city has fewer than three days, a notice explains that the cards show only the available days. The HTML uses the same display helpers as the main forecast, so units and number formatting stay consistent.

11. `public/style.css`: visual layout

Color and type variables

The `:root` block defines shared CSS custom properties. For example, `--page` and `--main-surface` are pale blue backgrounds, `--surface` is the warm sidebar, `--tile` is white, and `--ink` / `--muted` are text colors. Using variables means the palette can be changed in one place. Karla is the main font, with system fonts as fallbacks.

The universal `box-sizing: border-box` rule makes element widths include their padding and borders. The base `body` styles set the minimum screen size, font size, line spacing and page color. `button, input { font: inherit; }` keeps controls using the same typography as the rest of the page.

Desktop shell and sidebar

`.weather-shell` is a CSS Grid with a fixed-ish sidebar and a flexible main panel. `minmax(290px, 350px)` lets the sidebar grow within limits, and `minmax(0, 1fr)` gives the rest of the space to forecast content without forcing overflow.

`.side-panel` uses a cream background and column layout. Search styles define the form grid, input, button and hint. Focus rules draw a visible outline when the user tabs to a control. The today summary styles control the city label, weather icon, large average temperature and high/low values.

Main forecast and cards

`.main-panel` centers content vertically in the initial empty state and adds responsive padding. `.weather-results` constrains the content width. Forecast table styles set spacing, white row backgrounds, icon alignment and tabular number spacing. `.details-grid` puts the white packing and air cards next to each other when there is room.

The comparison section has a labeled form, a results area, two cards and a grid of forecast details. The card with the better weather gets a blue city name and label. Small separators between its daily rows make the list easier to scan.

Responsive rules

The three media queries change the layout as the screen narrows:

- At 1200px, the packing / air cards and comparison cards stack into one column.
- At 1000px, the shell changes from a two-column layout to a vertical layout; the sidebar moves above the main panel.
- At 650px, page padding and icon sizes shrink, the comparison form stacks its input and button, and daily comparison rows use a narrower grid.

These are ordinary CSS layout rules. There is no JavaScript device detection.

12. `public/weather-icons.svg`: reusable weather drawings

This file is one SVG document containing several `<symbol>` drawings: sun, moon, partly cloudy day/night, cloud, rain, storm, snow and mist. Each symbol has an `id` and a `viewBox`, which defines its coordinate system. Shapes are made from circles and paths with fills and strokes.

The page does not copy an icon's drawing every time. It uses `<svg><use href="/weather-icons.svg#rain"></use></svg>`. The `weatherIcon` method chooses the symbol name from OpenWeather's code. SVG icons remain sharp at the different sizes used for today, table rows and comparison rows.

13. The small learning server and project setup

server-first-iteration.js

This separate example keeps the first server flow short: read `req.query.city`, call Geocoding, take the first matching place, call Forecast with its coordinates, map each three-hour reading down to time, temperature and description, and send JSON.

It uses `express` and `async` / `await` like the main server. It listens on port 3001 so it can run separately from the app on port 3000. It deliberately leaves out input validation, error handling, daily grouping, air quality, packing advice and the comparison route. Read it as a learning sketch, not the complete application.

package.json

This file names the project, sets its entry point, gives the `npm start` command and lists Express as a dependency. The start command runs Node with `--env-file=.env`, so environment values are available to `process.env` while `server.js` starts. The project asks for Node 22 or newer because Node 22 supports this setup and has built-in `fetch`.

.env.example and .gitignore

`.env.example` shows the variable name the server expects, with a placeholder value. The real `.env` file contains the user's API key and is ignored by Git. It must stay local and should not be copied into a report or browser code.

`.gitignore` also excludes installed packages (`node_modules`), local design / agent folders, temporary assignment resources and the working report draft. These are local files that are not part of the source application.

README.md

The README tells a developer the Node requirement, how to install dependencies, where to put the key, how to start the server, which files make up the app, and what the two API routes do.

14. Important words and a simple way to trace a bug

- **CommonJS / `require`:** Node's module format used here to load Express and `path`.
- **Express:** a Node library for defining URL routes and serving static files.
- **Route:** a method and path such as `GET /api/weather` plus a function that handles it.
- **Request (`req`):** what the browser sent, including the query string.
- **Response (`res`):** what the server sends back, such as JSON and an HTTP status.
- **Promise:** an object representing a result that will arrive later, often after network I/O.
- `async` / `await`: syntax for writing Promise-based work in a readable order.
- **Event loop:** Node's mechanism for returning to other work while waiting for network operations.
- **JSON:** a text format for sending objects between the server and browser.
- **Vue state:** data returned from `data()` that Vue observes and uses to update the page.
- **Directive:** a Vue attribute such as `v-if`, `v-for`, `v-model` or `:class`.
- **Computed value:** a value such as `cityLabel` that Vue derives from reactive state.
- **Same origin:** the page and API use the same host and port, so the browser calls `/api/...` on the server that served the page.

When a feature fails, trace one step at a time: did the form method run? What URL did `fetch` request? Did the route run? What status and content type came back? What JSON shape did the server return? Which Vue value receives it? Which template section depends on that value?

For this app, an `Unexpected token '<'` JSON error usually means the browser received an HTML page from a static preview or stale server instead of the Express API. Run `npm start` in the project folder and open the app at `http://localhost:3000`.

Short recap

The server gets outside data and reshapes it into summaries. Vue stores that JSON as state. The HTML declares how each state should look. CSS controls layout and appearance. The SVG file supplies the weather symbols. Keeping those jobs separate is the main structure of this project.